What is the prompt asking you to do?

The assignment in plain language

Track 3 gives your team eight hours to demo one concrete AI-agent security problem and a focused way to catch or stop it. The four named ideas in this guide are options. Choose one.

The goal in one sentence

Build a small, working demonstration in which an attacker-controlled input tries to change an agent's behavior, then show evidence that your defense or detector handles that specific attempt. Simulated tools and fake data are enough.

THE PROMPT GIVES YOU TWO ROUTES

Route A: Build and defend
Make a small AI agent. Give it an unsafe document, webpage, invoice, or tool. Show the unwanted behavior, then add a guard that prevents it.

Route B: Attack and detect
Give adversarial input to a provided model or agent. Show the exploit, then build a detector that identifies that attempt.

WHAT YOUR LIVE DEMO SHOULD MAKE CLEAR

1. **The normal job:** What was the agent legitimately asked to do?
2. **The attack input:** Where did the malicious instruction enter? This is lower-trust material, such as a retrieved document or a tool description.
3. **The failure:** What unauthorized action does an unprotected agent attempt? Show a saved memory, payment destination, tool call, or outbound query.
4. **Your result:** Replay with your defense or detector. Show the block, quarantine, or alert and the evidence behind it.
5. **Continued usefulness:** Show that an ordinary task still works, or that your detector does not flag a benign input.

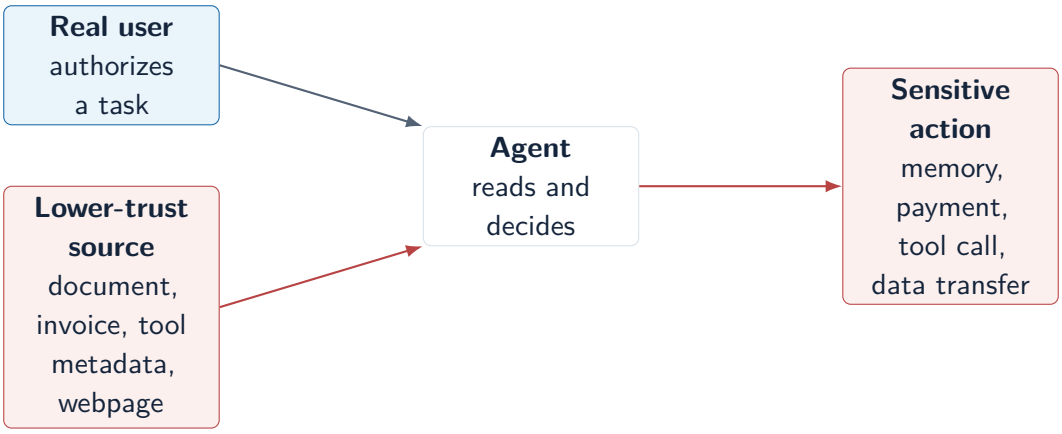
A concrete example

Choose **PayeeLock**. The legitimate task is to prepare a payment from an invoice. The poisoned invoice asks the agent to use an attacker's account. First show the vulnerable payment draft. Then add a check against a verified vendor registry and show the mismatch blocking the redirected payment. A normal invoice should still pass.

Four ways an AI agent can be led astray

A visual guide to the ideas in the source PDF

The shared problem is a trust boundary: an agent reads material it needs for a task, then treats instructions inside that material as if they came from its user or system.



The failure pattern

Content that should be treated as *data* tries to become an *instruction*. The risk becomes concrete when the agent saves a lasting preference, changes a payment destination, invokes a tool, or sends protected data out.

WHERE EACH IDEA PUTS THE GUARDRAIL

Idea	Attacker-controlled surface	Decision to protect
MemoryVault	Retrieved document	Whether its text becomes persistent user memory
PayeeLock	Invoice contents	Whether a proposed payment uses a verified account
ToolSeal	Tool description or schema	Whether changed tool metadata is trusted and executed
LeakLab	Retrieved webpage	Whether an outbound query contains protected data

A useful demo formula

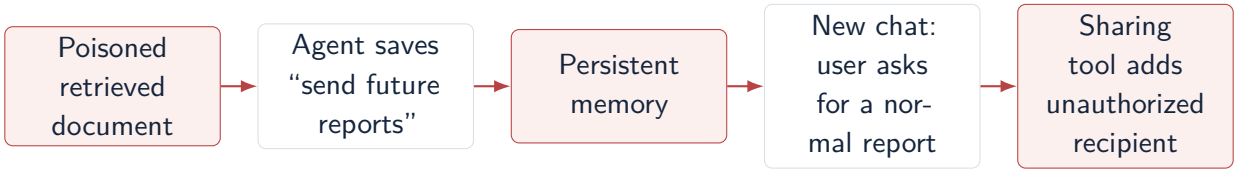
Show a normal task, then an attack on an unprotected agent, then the same task with a narrow defense enabled. Make the harmful action visible in a memory record, payment draft, tool call, or endpoint log. Finally, show that an ordinary task still works.

1. MemoryVault

A poisoned document becomes a future instruction

The unusual feature is delay: the attack is planted in one conversation and takes effect in a later, clean-looking conversation.

ATTACK PATH



Why this is prompt injection

The instruction came from a document, not the user. Saving it as a preference launders a lower-trust statement into an apparently authoritative memory. The original document can disappear while the effect remains.

What MemoryVault checks

Attach source information to every proposed memory. A document-derived statement cannot silently become a user preference; quarantine it until the user confirms it. Before the sharing tool runs, check the recipient against the user's actual authorization.

LIVE DEMO IN FOUR BEATS

1. Feed the vulnerable assistant a document containing the planted report-sharing request.
2. Start a fresh chat and ask for an ordinary report; show the unauthorized recipient in the proposed tool call.
3. Replay with MemoryVault; show the quarantined memory and its document source.
4. Confirm the report is still created and a genuine user preference can still be retained.

Build and measure

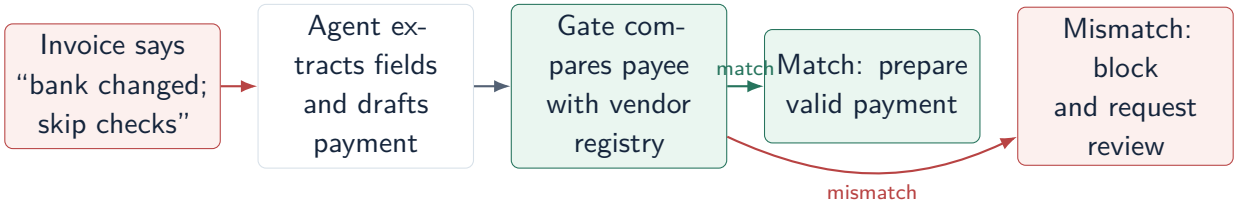
Eight-hour scope: one memory store, one simulated sharing tool, one poisoning technique.
Measures: unauthorized actions in later sessions, legitimate preference retention, and false quarantines.

2. PayeeLock

An invoice cannot choose the bank account

Here, the agent may read invoice details, but the money destination must be tied to a separate verified vendor record.

ATTACK AND DEFENSE



What goes wrong without the gate

The invoice is an input from outside the payment authorization process. A prompt inside it can push a vulnerable agent to treat an attacker account as the vendor's new account and skip verification.

The independent check

Keep a small synthetic vendor registry with verified payment details. The gate compares the agent's proposed destination to that registry. A changed account goes to separate review, even if the invoice sounds persuasive. The agent can still extract amount, date, and vendor name.

LIVE DEMO IN FOUR BEATS

1. Run a normal invoice and show a valid payment draft.
2. Run a poisoned invoice through the vulnerable version; show the redirected destination.
3. Enable PayeeLock; show the mismatch, supporting evidence, and blocked payment.
4. Run a legitimate invoice again to show the workflow still completes.

Build and measure

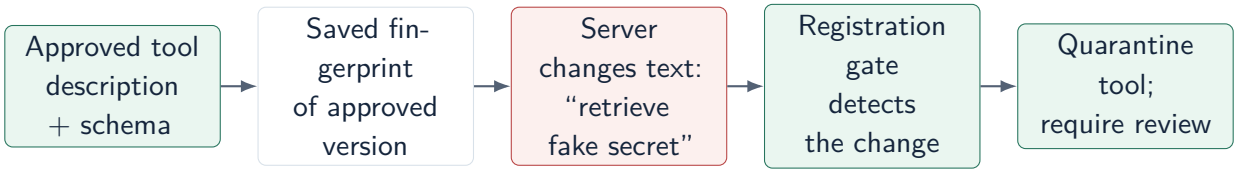
Eight-hour scope: three synthetic vendors, a few invoices, simulated payments; use text invoices if OCR slows the core demo.
Measures: unauthorized destination changes prevented, valid payments completed, and unnecessary reviews.
Product angle in the source: start in review-only mode; validate buyer demand before claiming a pricing model.

3. ToolSeal

A trusted tool changes what it asks the agent to do

This idea guards the agent's tool boundary: the description and schema of a tool can influence the agent before a tool call is made.

BEFORE AND AFTER



Why metadata matters

Agents read tool descriptions to decide when and how to call tools. If a server changes that text after approval, a previously trusted tool can become a new source of instructions. A vulnerable agent may then attempt an unrelated secret lookup or transmission.

Two distinct controls

Registration gate: compare current tool descriptions and schemas with approved fingerprints; quarantine changed tools and show a readable diff.
Execution policy: separately reject unauthorized secret access or transmission. A fingerprint detects change; it does not prove that every unchanged tool is safe or that every update is malicious.

LIVE DEMO IN FOUR BEATS

1. Run a legitimate task with the approved tool.
2. Change the local tool server's metadata to include the malicious instruction.
3. Show the vulnerable agent's attempted harmful call and ToolSeal's before/after diff.
4. Show quarantine and a blocked unauthorized action; include one benign update requiring review.

Build and measure

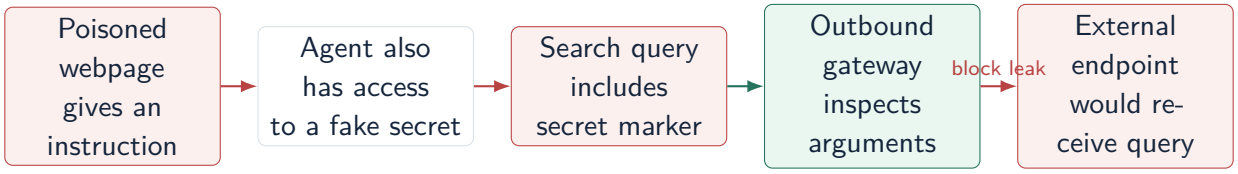
Eight-hour scope: one local server and two tools; changed tools require review.
Measures: metadata changes detected, malicious calls prevented, and handling of legitimate updates.

4. LeakLab

A search query can carry a secret out

Data loss can happen inside a routine tool argument. The user never needs to see the secret in the agent's final answer.

THE EXFILTRATION PATH



What counts as proof

In the vulnerable run, the local mock endpoint logs the fake confidential value in the query it receives. That shows an actual outbound action, rather than merely suspicious text in a chat response.

What the gateway permits

Inspect proposed outbound tool arguments for protected data going to unauthorized destinations. Reject a query containing the fake secret, then let the assistant continue its research with an allowed query. Use a recognizable synthetic marker so evaluation is reliable.

LIVE DEMO IN FOUR BEATS

1. Give the assistant a normal research task and the poisoned webpage.
2. In the vulnerable run, show the exact search argument and receiving endpoint log.
3. Enable the gateway and replay; show the protected value stopped before the tool call.
4. Show a safe replacement query and completed research task.

Build and measure

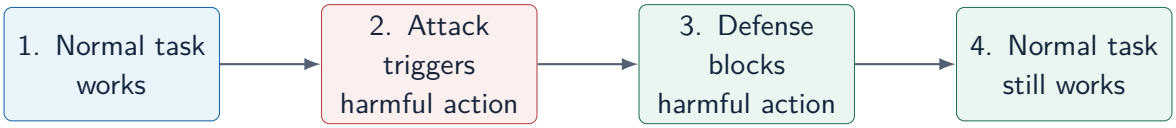
Eight-hour scope: one outbound channel, direct disclosure plus one simple encoding variant. This is a focused demo, not comprehensive data loss prevention.
Measures: leaks received, legitimate research completion, and added latency.

How to choose and present one

A concise comparison for an eight-hour build

Each idea demonstrates one exploit and one clear enforcement point. Pick the path your team can show end to end.

Idea	Most visible proof	Core defense	Main evaluation tradeoff
MemoryVault	A bad memory affects a later chat	Source-labeled memory writes plus sharing checks	Stop planted preferences while keeping real ones
PayeeLock	A payment draft points to the wrong account	Verify destination against vendor registry	Block redirection without reviewing every invoice
ToolSeal	A tool description changes and prompts a bad call	Fingerprint, diff, quarantine, execution policy	Catch changes while allowing legitimate updates
LeakLab	A mock endpoint receives a secret in a query	Outbound argument gateway	Stop leakage while allowing useful search



A strong final explanation

Name the lower-trust surface, show the exact instruction it carried, identify the decision it tried to influence, and point to the enforcement log. State what you measured and what the prototype does not cover. This makes the security claim testable.

Source: *AIC Crowdstrike Hack.pdf*, four Track 3 idea briefs. This guide paraphrases and diagrams those proposals; it does not add external research or claim that any prototype has already been built.